Aula 02 - Classes e Objetos em Java

Construtores, Sobrecarga, `this` e Métodos Estáticos

Prof. Dr. Welington Galvão Rodrigues

Análise e Desenvolvimento de Sistemas - ADS
IFMA — Campus Grajaú

12 de agosto de 2026

INSTITUTO FEDERAL
Maranhão

INSTITUTO FEDERAL
Maranhão

Agenda

1. Retomada da Aula 01
2. Construtores
3. Sobrecarga de Construtores
4. A Referência this
5. Métodos de Instância x Estáticos
6. Live Coding: ContaBancaria
7. Síntese e Próxima Aula

INSTITUTO FEDERAL
Maranhão



Retomada da Aula 01

Antes de Começar...

- **"O que diferencia uma classe de um objeto?"**
- **"O que é um atributo? E um método?"**
- **"Alguém teve dificuldade para instalar o JDK ou a IDE?"**

Agenda de hoje
Como garantir que um objeto **nasça com dados corretos**, e não "quebrado"? Construtores, sobrecarga, `this` e métodos estáticos.



Construtores

O Bug do Objeto "Quebrado"

```java
RegistroAcademico michael = new RegistroAcademico();
RegistroAcademico roberto = new RegistroAcademico();

michael.inicializa("Michael Goodrich", 34980030, (byte) 2, 100.0);

System.out.println("Mensalidade do Michael: " + michael.
    calculaMensalidade());
System.out.println("Mensalidade do Roberto: " + roberto.
    calculaMensalidade());
```

Listing 1: Uma instância inicializada, outra não

Saída: `500.0` e `NaN`

O Bug do Objeto "Quebrado" (cont.)

Por que "NaN"?

O compilador **não obriga** o uso de métodos de inicialização — só obriga o uso de `new`. Os campos de `roberto` ficaram com valor *default* zero, e o cálculo envolveu uma divisão $0/0$.

O que é um Construtor?

Construtor — método especial, chamado **automaticamente** sempre que uma instância é criada com `new`.

Regras

- Mesmo nome da classe (maiúsculas/minúsculas inclusive)
- Não tem tipo de retorno — nem mesmo `void`
- Não pode ser chamado diretamente pelo programador usuário

Diferente de um método comum

O programador **não pode esquecer** de chamar o construtor: ele roda antes de qualquer outro código da instância.

Valores *Default*

Quando um campo **não é inicializado**, Java atribui um valor padrão automaticamente:

Tipo	Valor
`boolean`	`false`
`char`	(espaço)
inteiros / ponto flut.	`0`
referências (`String`...)	`null`

Construtor *Default* do Compilador

- Se **nenhum** construtor for declarado, o compilador cria automaticamente um construtor **sem argumentos e sem código**;
- Equivale a escrever `Televisor() { }`;
- Assim que o programador declara **ao menos um** construtor, esse construtor *default* deixa de ser gerado.

Atenção
Se você declarar só um construtor com argumentos, `new Classe()` **(sem argumentos)** deixa de compilar — a não ser que você também declare esse construtor explicitamente.

Corrigindo com um Construtor

```
class EventoAcademico {
    String nomeDoEvento, localDoEvento;
    Data inicioDoEvento, fimDoEvento;
    int numeroDeParticipantes;

    // construtor na sequência...
}
```

Listing 2: EventoAcademico: atributos

Corrigindo com um Construtor — Construtor

```java
class EventoAcademico {
    // ...atributos já declarados

    EventoAcademico(String n, String l, Data i, Data f, int num) {
        nomeDoEvento = n;
        localDoEvento = l;
        inicioDoEvento = new Data();
        inicioDoEvento.inicializaData(i.retornaDia(), i.retornaMes()
    , i.retornaAno());
        // ...
        numeroDeParticipantes = num;
    }
}
```

Listing 3: EventoAcademico exige os dados na criação

Corrigindo com um Construtor (cont.)

Ponto de atenção

Instâncias de outras classes usadas como campos (aqui, `Data`) também devem ser inicializadas dentro do construtor — copiar só a referência faria duas instâncias apontarem para o **mesmo** objeto interno.



Sobrecarga de Construtores

Por que Sobrecarregar Construtores?

Situação real: nem sempre queremos exigir todos os dados na criação de um objeto.

- Um robô pode nascer numa posição específica **ou** na origem $(0, 0)$, olhando para o Norte;
- Um televisor pode ser criado já sintonizado **ou** com valores padrão.

Pergunta: *"Poderíamos ter métodos com nomes diferentes para cada forma de criar o objeto, como* `criarRoboComPosicao`*?"*

> **Não, para construtores**
> Construtores **precisam** ter o mesmo nome da classe. A solução é a **sobrecarga**.

Sobrecarga e Assinatura de Métodos

Sobrecarga (*overloading*): vários métodos/construtores com o **mesmo nome**, desde que suas **assinaturas** sejam diferentes.

> **Assinatura de um método**
> Nome + tipos (quantidade e ordem) dos parâmetros. **O tipo de retorno não faz parte da assinatura.**

O compilador decide, em tempo de compilação, **qual versão chamar**, com base nos argumentos passados.

Exemplo: Três Construtores para `RoboSimples`

```
class RoboSimples {
    String nomeDoRobo;
    int posicaoXAtual, posicaoYAtual;
    char direcaoAtual;

    RoboSimples(String n, int px, int py, char d) { /* completo */ }

    // demais construtores na sequência...
}
```

INSTITUTO FEDERAL
Maranhão

Exemplo: Três Construtores para `RoboSimples` — Encadeados

```java
class RoboSimples {
    // ...campos e construtor completo já declarados

    RoboSimples(String n) {
        this(n, 0, 0, 'N');
    }

    RoboSimples() {
        this("", 0, 0, 'N');
    }
}
```

Exemplo: Três Construtores para `RoboSimples` (cont.)

Exemplo

```
new RoboSimples("Groucho", 20, 10, 'S')  |  new RoboSimples("Chico")  |  new RoboSimples()
```

Exemplo: `Televisor` com Três Construtores

Construtor	Uso
`Televisor()`	Valores *default* (volume e canal em 0)
`Televisor(int volume)`	Volume definido; canal em 0
`Televisor(int volume, int canal)`	Volume e canal definidos

```
1 Televisor tv1 = new Televisor();
2 Televisor tv2 = new Televisor(150);
3 Televisor tv3 = new Televisor(150, 3);
4
```

Cuidado: Promoção x Rebaixamento de Tipos

- Java **promove automaticamente** tipos menores para maiores sem perda de informação (`byte` → `int`);
- Java **nunca rebaixa automaticamente** (`double` → `int`) — exige *cast* explícito.

Não compila

`new RoboSimples("Rosie", 10.0f, 22.0f, 'S')` — coordenadas `float` não podem virar `int` automaticamente.



A Referência this

O Problema do Nome Repetido

“Quando você aponta para si mesmo e diz "eu", não precisa dizer seu próprio nome — `this` é exatamente essa autorreferência.”

— Analogia da aula

```
1 public void setCanal(int canal) {
2     canal = canal;    // isto NAO funciona como esperado!
3 }
```

Pergunta: *"Por que essa atribuição não altera o atributo `canal` da classe?"*

A Palavra-Chave `this`

`this` faz referência ao **objeto corrente** — o objeto que chamou o método ou está sendo construído.

Quando o parâmetro tem o **mesmo nome** de um atributo, as duas ocorrências dentro do método são entendidas como a variável **local** (o parâmetro).

```
1 public void setCanal(int canal) {
2     this.canal = canal;  // this.canal = atributo | canal =
      parametro
3 }
```

`this` Resolvendo Ambiguidade

```java
public class Televisor {
     int volume;
     int canal;

    public Televisor(int volume, int canal) {
        this.volume = volume;
        this.canal = canal;
    }
}
```

Convenção comum em Java

Manter o nome do parâmetro igual ao do atributo (usando `this` para diferenciá-los) evita ter que inventar nomes artificiais como `v` ou `novoVolume`.

this(...) Encadeando Construtores

```
1  RoboSimples(String n, int px, int py, char d) {  // "completo"
2      nomeDoRobo = n; posicaoXAtual = px;
3      posicaoYAtual = py; direcaoAtual = d;
4  }
5
6  RoboSimples(String n) {
7      this(n, 0, 0, 'N');    // chama o construtor completo
8  }
9
10 RoboSimples() {
11     this("", 0, 0, 'N');  // encadeamento em cascata
12 }
```

Regras para `this(...)`

1. Só um construtor pode chamar outro construtor;
2. A chamada `this(...)` deve ser a **primeira linha**;
3. Não se chama um construtor pelo nome da classe — só por `this(...)`;
4. Um construtor **pode** chamar métodos comuns;
5. Não há chamadas recursivas de construtor.

Vantagem
Menos duplicação de código: mudou a lógica de inicialização? Basta alterar o construtor "completo".



Métodos de Instância x Estáticos

Instância x Estático

Pergunta: *"Por que `Math.sqrt(9)` não precisa de um objeto `Math`, mas `tv1.ligar()` só faz sentido depois de criarmos `tv1`?"*

Método de instância
Pertence a um **objeto específico**; opera sobre o estado daquele objeto; chamado via `objeto.metodo()`.

Método estático (`static`)
Pertence à **classe**; chamado via `Classe.metodo()`; **sem** acesso a `this` nem a atributos de instância.

Tabela Comparativa

Aspecto	Instância	Estático
Pertence a	um objeto	a classe
Chamada	`objeto.m()`	`Classe.m()`
Acessa atributos	sim (via `this`)	só `static`
Exemplo	`conta.depositar()`	`Math.sqrt(9)`

Tabela Comparativa — Exemplo em Código

```java
public class ContaBancaria {
     static int totalDeContas = 0;    // compartilhado
     double saldo;                     // por objeto

    public void depositar(double valor) { saldo += valor; }
    public static int getTotalDeContas() { return totalDeContas; }
}
```



Live Coding: ContaBancaria

Atributos da Classe

```java
public class ContaBancaria {

    static int totalDeContas = 0;  // compartilhado por todas as contas

    String titular;
    int numero;
    double saldo;

    // construtor na sequência...
}
```

Construtor Completo

```java
public class ContaBancaria {
    // ...atributos já declarados

    public ContaBancaria(String titular, int numero, double saldo) {
        this.titular = titular;
        this.numero = numero;
        this.saldo = saldo;
        totalDeContas++;
    }
}
```

Sobrecarga com `this(...)`

```java
public ContaBancaria(String titular, int numero) {
    this(titular, numero, 0.0);   // saldo inicial sempre zero
}
```

Para a turma

Faz sentido um construtor `ContaBancaria()` sem argumentos? Que valores *default* ele usaria para `titular` e `numero`?

Métodos de Instância e Método Estático

```java
public void depositar(double valor) { saldo += valor; }

public void sacar(double valor) {
    if (valor <= saldo) saldo -= valor;
    else System.out.println("Saldo insuficiente.");
}

public double consultarSaldo() { return saldo; }

public static int getTotalDeContas() { return totalDeContas; }  //
    estatico
```

INSTITUTO FEDERAL
Maranhão

Testando a Classe

```java
ContaBancaria c1 = new ContaBancaria("Maria Silva", 1001, 500.0);
ContaBancaria c2 = new ContaBancaria("Joao Pereira", 1002);  // saldo 0.0

c1.depositar(200.0);
c2.depositar(100.0);
c2.sacar(30.0);

System.out.println(c1);
System.out.println(c2);
System.out.println("Total de contas: " + ContaBancaria.getTotalDeContas());
```

INSTITUTO FEDERAL
Maranhão

Testando a Classe — Saída

Saída:

```
Conta 1001 (Maria Silva) -- saldo:  R$ 700.0
Conta 1002 (Joao Pereira) -- saldo:  R$ 70.0
Total de contas:  2
```

Exercício em Sala

Classe `Pessoa` (em duplas)

Crie uma classe `Pessoa` com atributos `nome`, `idade` e `cpf`, contendo:

- ao menos **dois construtores** sobrecarregados;
- uso de `this` para resolver ambiguidade de nomes;
- um **método estático** simples (ex.: contador de pessoas cadastradas).



Síntese e Próxima Aula

Conceitos-Chave desta Aula

- Construtor: inicializa o objeto via `new`
- Construtor *default*: só sem construtores declarados
- Valores *default*: `false`, `0`, `null`
- Sobrecarga: mesmo nome, assinaturas diferentes
- Assinatura: nome + tipos dos parâmetros
- `this`: autorreferência ao objeto atual
- `this(...)`: encadeia construtores
- Instância x estático: objeto x classe

Próximas Aulas

Aula 03 — 22/08/2026 (sábado, reposição de 19/08)
Encapsulamento: modificadores de acesso (`private`, `public`, `protected`), *getters* e *setters*, validação de dados.

Aula 04 — 26/08/2026 (quarta-feira)
Abstração e relacionamentos entre objetos: associação, agregação e composição.

Encaminhamentos

Entregar a lista de exercícios 1 ao final da aula. Leitura complementar: SANTOS, R. — Cap. 4 (exercícios 4.1 e 4.2).

Obrigado!

Dúvidas ou sugestões?

welington.rodrigues@ifma.edu.br



INSTITUTO
FEDERAL
Maranhão

Aula 02 - Classes e Objetos em Java

Construtores, Sobrecarga, `this` e Métodos Estáticos

Prof. Dr. Welington Galvão Rodrigues

Análise e Desenvolvimento de Sistemas - ADS
IFMA — Campus Grajaú

12 de agosto de 2026

INSTITUTO FEDERAL
Maranhão

INSTITUTO FEDERAL
Maranhão